PRANA

Offline-First Edge-AI Personal Health Companion

SIH Judge Technical Q&A Reference — expected questions with code-accurate, defensible answers spanning the current build, the future roadmap, and known risks and limitations.

How to use this: answers are written the way you should say them out loud. Where the app uses a heuristic (not a trained model) or a demo-only value, the answer states it plainly. Never overclaim — a judge who catches one bluff distrusts every other answer.

1. Product & Problem Statement

The 30-second pitch and why the design choices are what they are.

Q In one sentence, what is PRANA?

PRANA is an offline-first, edge-AI personal health companion: a phone app paired with a wearable band that reads your vitals and environment, learns your personal baseline, and gives you a live health-risk score with plain-language advice and an automatic fall/SOS pathway — all on-device, with no internet required.

Q What problem does it solve?

India faces climate-health risks — heatwaves, air pollution, floods, cyclones — that hit vulnerable people (elderly, outdoor workers, those with existing conditions) hardest, often in areas with poor connectivity. PRANA turns consumer wearable signals into an early-warning and self-care tool that keeps working with no network, and escalates to an emergency contact automatically when it detects a fall.

Q Why offline-first? Isn't the cloud easier? HONESTY

Two reasons. First, our target users are exactly where connectivity is weakest — during a disaster the network is often the first thing to fail. Second, privacy: health data never leaves the phone, which sidesteps a large class of consent, storage and breach concerns. Everything — feature extraction, the risk model, recommendations — runs on the device.

Q Who is the user, and what does the vulnerable-profile flag do?

The UserProfile carries name, age, gender, height, weight, blood group, medical conditions, emergency contacts and an "isVulnerableProfile" flag. That flag lets the app be more conservative for elderly or high-risk users and drives the dedicated "vulnerable elderly" demo scenario.

2. Architecture

How the app is put together — state, structure, and separation.

Q **Walk me through the architecture.**

It is a Flutter app using Provider + ChangeNotifier for state. A single PranaHealthController is the composition root: it owns the sensor source, baseline service, risk engine, ML inference service, recommendation engine, emergency service and local storage, wired by constructor injection. The UI is feature-first (dashboard, health/vitals, insights, environment, emergency, report, profile), and screens listen to the controller. Sensor data flows one way: source -> controller -> feature vector -> ML -> risk engine -> UI.

Q **That controller sounds like a "god object". Is that a problem?** HONESTY

Fair observation — it is a large orchestrator (~700 lines) and that is a known refactor target. We accepted it deliberately for the hackathon: a single, readable wiring point made the data flow easy to reason about and demo. The individual services (baseline, risk, ML, emergency) are already separated and independently testable; the next step is to split the controller into smaller feature controllers or a bloc-style layer.

Q **How do you talk to native Android features without plugins?**

We use Flutter MethodChannels implemented directly in our own app module (MainActivity), not third-party native plugins. There is a "prana/emergency" channel for SMS, phone calls and GPS, and a "prana/files" channel for sharing the generated PDF via Android FileProvider. This was a deliberate choice — adding native plugins under the current Android Gradle Plugin toolchain was fragile, so we kept native code inside our own module and used pure-Dart packages (pdf) everywhere else.

Q **What is the tech stack, concretely?**

Flutter/Dart; Provider for state; flutter_blue_plus for BLE; tflite_flutter is on the classpath for the neural path; the pure-Dart pdf package for report generation; shared_preferences for persistence; intl for formatting. The wearable firmware is an ESP32 sketch (Arduino/C++) driving a MAX30102 pulse-ox, an MPU6050 IMU and a DHT22 temp/humidity sensor.

3. AI / ML Pipeline

The most likely area for hard questions — answer it precisely.

Q **What "AI" actually runs when the app is live?** HONESTY

Be precise here. The live inference path is a set of five hand-tuned logistic-regression classifiers (one each for heat stress, respiratory, cardiovascular, fatigue, and an overall anomaly score). Each is a sigmoid over a weighted sum of our 16-feature vector, with coefficients set from published physiological thresholds. It runs fully on-device, no network. It is a defensible, explainable heuristic model — not a network-trained classifier fitted to a labelled dataset. We call the class MockInferenceService, but the math is real and runs for real; "mock" means it stands in for the TFLite runner.

Q You ship a .tflite file. Is the neural network being used?

HONESTY

Honest answer: not in the live path. We have a trained denoising autoencoder (prana_autoencoder_best.tflite) built from public datasets — BIDMC, MMASH and PPG-DaLiA, roughly 97k rows — and a working TFLite inference service that loads it. But when we wired it in, the autoencoder collapsed: it copied one saturated anomaly score across all domains and pinned risk to "critical". Rather than ship a broken model, we fell back to the transparent logistic heuristics for the demo and kept the TFLite path in the codebase for the roadmap.

Q So why keep the neural model at all?

FUTURE

Because the pipeline around it is real and reusable: a frozen 16-dimension feature contract (versioned 1.1.0-frozen), a feature processor, a masking scheme for unavailable signals, and a P95-based anomaly threshold. The roadmap is to retrain the autoencoder (or move to a supervised model) on wrist-collected data, validate it against the heuristic, and swap it in behind the same interface — the app would not change, only the InferenceService implementation.

Q What is in the 16-feature vector?

It blends live vitals (HR, SpO2, body temp), environment (heat index, AQI, humidity), personal baseline deviations (HR % deviation, SpO2 drop, temp elevation), activity/exposure, and an HR-trend slope. That slope (feature 15) is a genuine least-squares fit over recent HR history and returns 0.0 when there are too few real samples — it does not fabricate a trend. Unavailable signals are masked rather than faked.

Q Where does inference run and how fast?

FUTURE

On the main isolate, once per incoming reading. On real hardware that is roughly one health packet per second; in demo mode every two seconds. The logistic model is a handful of multiply-adds, so latency is negligible. If we move to a heavier neural model we would push inference onto a background isolate — that is a known, planned change.

4. Risk Engine

The scoring math — know the weights and thresholds cold.

Q How is the 0-100 risk score computed?

The engine builds four domain subscores (each 0-1): heat stress, respiratory, cardiovascular and autonomic fatigue. Each blends the ML probability with rule-based environmental and physiological factors for example heat stress is 0.5*ML + 0.3*(heat-index factor) + 0.2*(body-temp elevation). The final score is 0.50*max-domain + 0.30*mean-domain + 0.20*anomaly-probability, times 100, clamped 0-100. Weighting the max domain highest means one genuinely dangerous system drives the alert even if the others look fine.

Q What do the risk levels mean?

Below 30 is Safe, 30-59 Moderate, 60-79 High, 80 and above Critical. Those thresholds map to the colour states and advice the user sees.

Q How is the trend arrow (Spiking/Increasing/Stable/Improving) derived? HONESTY

It is the difference between the current score and the single previous score: greater than +8 is "Spiking", greater than +2 "Increasing", less than -3 "Improving", otherwise "Stable". Be honest that this is a two-sample delta, not a windowed slope, and it resets on app restart a smoothed multi-sample trend is on the list.

Q How does the app explain a score to the user?

The engine produces DriverFactors the top three contributing causes ranked by impact (e.g. elevated heart rate, heat stress, poor air quality) and the RecommendationEngine emits a primary and secondary action via a priority cascade (heat -> respiratory -> cardiovascular -> fatigue -> moderate fallback), with live vitals interpolated into the text. So the user sees why, not just a number.

Q Your RiskResult reports a confidence of 0.92. Is that real? HONESTY

No that is a fixed placeholder, not a computed uncertainty. I would not present it as a real confidence interval. Producing a genuine calibrated confidence is part of the ML roadmap; today the honest framing is "explainable score", not "statistically calibrated probability".

5. Baseline Learning

Personalisation what "learns your normal" actually means.

Q What does "learns your personal baseline" mean technically?

The BaselineService keeps rolling buffers (last 200 samples) of resting heart rate, SpO2 and body temperature, and only accepts readings taken at rest or low activity so exertion does not skew it. Every 20 accepted samples it recalibrates the normal ranges using a rolling mean and standard deviation (a 90% bound, mean +/- 1.645 sigma, with sensible floors and caps). It is honest statistics, not a black box.

Q Is that an EWMA or percentile model?

HONESTY

No — it is a rolling sample mean and standard deviation over the last 200 resting samples, recalibrated in batches of 20. I would not call it adaptive filtering. It is deliberately simple and inspectable; EWMA or percentile bands are an easy future upgrade behind the same interface.

Q What is the "learning — 60 readings" bar on the dashboard?

That is a UI progress indicator, not an engine threshold. Each recalibration credits 20 readings, so the baseline is treated as "fully learned" after 3 recalibrations = 60 accepted resting samples. Until then the app shows it is still calibrating and leans on sensible default priors (HR 65-88, SpO2 floor 95, temp 36.4-37.3).

Q Does the baseline learn in the demo you are about to show?

HONESTY

Important caveat: no. Baseline learning is disabled in mock/demo mode by design, so simulated data never corrupts a real user's learned normal. In the demo the deviations are computed against the default priors. On real hardware it learns continuously.

6. Sensors, Wearable & BLE

What the band measures, what the phone estimates.

Q What does the band actually measure?

HONESTY

On the reference ESP32 firmware: heart rate and SpO2 from a MAX30102 pulse-ox (~1 Hz, and only sent when a finger is detected and values are physiologically sane), 3-axis motion, step count, activity label and fall events from an MPU6050 IMU at 50 Hz, and ambient temperature + humidity from a DHT22. Body temperature from an IR sensor is optional and off by default — when absent the app falls back to a default and does not pretend to measure it.

Q Which values are measured versus estimated on the phone?

HONESTY

Measured: HR, SpO2, motion/steps/falls, ambient temp/humidity. Computed on-phone: the heat index (Rothfusz regression), all risk subscores, the anomaly score, recommendations and trend. Not truly measured: HRV/RMSSD (it is a synthetic constant in the current paths and the ML mask even flags it unavailable), and resting HR over the standard GATT path echoes current HR. AQI comes from the packet if present — the reference band has no air-quality sensor — otherwise from the demo generator.

Q How does the phone talk to the band?

Over Bluetooth Low Energy with flutter_blue_plus. There is a custom GATT service with a JSON-telemetry characteristic (notify), plus standard SIG characteristics for HR, SpO2, health thermometer and battery, and a custom IMU characteristic. It requests a 256-byte MTU for the JSON payloads and scans for a device named PRANA_BAND_PRO (with a PRANA_ prefix fallback). Crucially, the parser only emits a stream when the packet truly contains those keys, so we never invent a 75-BPM default from a malformed packet.

Q What if there is no band? Can you still demo?

Yes — there is a MockSensorDataSource that generates realistic vitals at 0.5 Hz by smoothly approaching per-scenario targets with small noise. The app picks mock versus Bluetooth from a stored setting and can switch at runtime. That is what powers the disaster scenarios in the live demo.

7. Emergency & Fall Detection

The safety-critical path — precise numbers matter here.

Q How does fall detection work?

It is layered. The band is the real detector: at 50 Hz it looks for free-fall (below ~0.55g), then an impact (above 2.8g, guarded so a table knock is rejected), then stillness held for ~1.2s within a 2.5s window, with a 15s refractory lockout. The phone has a lighter backstop — a 2.8g spike followed by near-stillness (below 0.25g) within 2 seconds — and it also honours the hardware's fall flag directly. Detection lives on the band because an impact peak lasts 100-300 ms and would fall between the phone's packets.

Q What happens after a fall is detected?

The EmergencyService starts a 10-second countdown so the user can cancel a false alarm. If it reaches zero, the controller dispatches once: it resolves GPS, builds an alert message with the user's identity, vitals and a Google Maps location link, sends an SMS to the primary emergency contact, then places a phone call to them. SMS, call and location go through our native prana/emergency channel.

Q Does this work when the phone is locked or in the background? RISK

Honestly, not reliably today. The app has no Android foreground/background service, so when the screen is locked the OS can suspend BLE streaming and fall processing. The fix is a native foreground service with a persistent notification so sensing and the SOS pathway survive a locked screen — that is the single most important reliability item on the roadmap for a safety app.

Q What if GPS or SMS permission is missing?

Every native call is wrapped so a denied permission or a non-Android platform degrades gracefully rather than crashing — location returns null, SMS/call return a failure status, and the UI falls back to a manual "copy this message" path. The Emergency screen also surfaces which permissions are missing so the user is warned before they need it.

8. Data Privacy & Storage

Answer this carefully — do not claim encryption you do not have.

Q Where is health data stored and is it encrypted?

HONESTY

All data stays on the device — there is no backend and no network inference. Be honest about the current storage: day-level summaries and the baseline are persisted in SharedPreferences as plaintext JSON. We have the dependencies for an encrypted SQLCipher/Drift store and secure-storage on the classpath, but that path is not active yet. So today the honest claim is "never leaves your phone", not "encrypted at rest" — activating the encrypted store is a near-term item.

Q What exactly is persisted, and for how long?

Only daily aggregates (average/peak HR, minimum SpO2, peak temp, average/peak risk, steps, alert counts) for up to 90 days, plus the learned baseline and the user profile. Raw minute-by-minute vitals are never written to disk — the controller keeps only a small in-memory buffer of recent points for the live charts. Demo-mode days are explicitly excluded so simulated data never enters the real record.

Q Does any data ever leave the device?

Only when the user initiates it, and only two paths: the emergency SMS/call to a contact the user configured, and the doctor-report PDF which the user chooses to share. There are no analytics, no telemetry, no cloud sync — and there is no INTERNET permission wired for data upload.

Q Is this HIPAA / medical-grade compliant?

HONESTY

No, and we do not claim to be. PRANA is a wellness and early-warning aid built on consumer-grade sensors, not a regulated medical device. Formal compliance, clinical validation and certification would be required before any diagnostic claim — that is explicitly out of scope for the current prototype.

9. Disaster Scenarios & Demo

How the live demo is driven, and what is scripted.

Q What are the disaster scenarios and how are they driven?

There are six: normal, heatwave, air pollution, flood, cyclone and vulnerable-elderly. Selecting one, in demo mode, changes the target vitals and environment the mock generator converges toward — e.g. heatwave pushes ambient to ~41.5 deg C, humidity 72%, HR ~108, SpO2 ~94, body temp ~38.2. On real hardware the selection is recorded but does not alter the live sensor stream. This lets us show the full risk range on stage without a heat chamber.

Q Is the demo just a scripted animation?

No. The scenario only sets the target inputs; everything downstream — feature extraction, the logistic risk model, the composite score, drivers and recommendations — runs the exact same code that runs on real sensor data. Swap the mock source for the Bluetooth source and the pipeline is unchanged.

Q Can you trigger a fall in the demo safely?

Yes — there is a test-fall trigger. In mock mode it injects a high-G motion sample with the fall flag set, which enters the same detection and countdown path a real fall would; on hardware it calls the emergency service directly. So we can demonstrate the full SOS flow on demand.

10. Future Roadmap

Show ambition grounded in what is already scaffolded.

Q What is the immediate next step after the hackathon? FUTURE

Three things, in order: (1) a native foreground service so sensing and SOS survive a locked screen — critical for a safety app; (2) activate the encrypted on-device store that is already scaffolded; (3) retrain and validate the neural model against the heuristic and swap it in behind the existing inference interface.

Q How would you validate the model clinically? FUTURE

Collect wrist-worn data with reference-device labels, evaluate the heuristic and the retrained model against those labels for sensitivity/specificity per domain, and calibrate a real confidence rather than the current placeholder. Only after that would we make any stronger claim than "early-warning aid".

Q How does this scale to real deployment? FUTURE

The on-device design scales naturally — there is no server cost per user, and it works in low-connectivity regions. For population-level insight we could add opt-in, anonymised, aggregated uploads (never raw vitals) for public-health heat/air-quality mapping, gated behind explicit consent. An optional caregiver-dashboard companion is a natural extension of the emergency-contact feature.

Q Could this integrate real weather / AQI feeds?

FUTURE

Yes — that is a clean extension. Because environment readings flow through one normalised model, a keyless public feed (e.g. Open-Meteo) could supplement or replace the band's ambient readings for forecast-based warnings, added as an isolated, read-only data source without touching the risk engine. It is deliberately not in the current build to keep the offline-first guarantee clean.

11. Risks, Limitations & Tough Questions

Pre-empt the hardest questions with candour.

Q What is the single biggest limitation of the current build?

RISK

The locked-screen reliability gap: with no foreground service, continuous sensing and the automatic SOS are not guaranteed when the phone is locked. For a safety product that is the top priority to fix, and I would rather name it than have you find it.

Q Aren't consumer wearables too inaccurate for health decisions?

HONESTY

They have real limits, and we design around them. Wrist/finger PPG SpO2 and HR are motion-sensitive and need stable contact — so the firmware withholds vitals when the signal is unstable rather than reporting noise. There is no ECG, no medical-grade pulse-ox and no true HRV. That is exactly why we position PRANA as an early-warning and self-care aid, not a diagnostic device, and keep a human-cancellable countdown before any emergency action.

Q What happens with false positives — a false fall alert?

RISK

The 10-second cancellable countdown is the primary guard, and the hardware detector is specifically tuned to reject ordinary knocks (it requires a free-fall or a very high peak before an impact counts). False negatives are the more serious risk for a safety device, which is why detection runs at 50 Hz on the band rather than relying on the slower phone stream.

Q If the ML is "just heuristics", what is genuinely novel here?

HONESTY

The integration. An offline, explainable, personalised risk-fusion pipeline that combines vitals, environment and a learned baseline; a hardware+phone layered fall detector tuned for real impact physics; and an automatic, permission-graceful SOS with location — all working with no network and no data leaving the phone. The model is swappable by design; the system around it is the contribution.

Q What are the known bugs or rough edges you would fix first?

HONESTY

The encrypted-storage stack is scaffolded but not active; the release build currently uses debug signing; the trend is a two-sample delta; confidence is a placeholder; and the controller wants decomposition. None of these affect the demo path, and each is a scoped, known fix rather than an unknown risk.